



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

پروژه درس معماری کامپیوتر

تحلیل نقصان‌های برخورد و طراحی معماری حافظه نهان قربانی

نگارش

نفیسه زارعی

آرش اکبری

آیناز رحمانی

استاد

دکتر حمید سربازی آزاد

مرداد ۱۴۰۵

چکیده

در این پروژه به بررسی و حل مشکل افت عملکرد ناشی از نقصان‌های برخورد در حافظه‌های نهان با روش نگاشت مستقیم پرداخته شده است. تداخل آدرس‌ها در این ساختار منجر به پدیده تلاطم و مراجعه‌های مکرر و پرهزینه به حافظه اصلی می‌شود. برای رفع این چالش، یک حافظه نهان قربانی تمام‌انجمی طراحی و بین سطح اول حافظه نهان و سطح پایین‌تر قرار داده شد. این معماری در شبیه‌ساز آکیتا پیاده‌سازی شده و عملکرد آن با استفاده از یک برنامه محک با دسترسی‌های متناوب مورد ارزیابی قرار گرفت. نتایج شبیه‌سازی و بررسی شمارنده‌های سخت‌افزاری نشان می‌دهد که افزودن این پیمان به مدیریت صحیح جابه‌جایی دوطرفه، ترافیک حافظه اصلی را به شکل چشمگیری کاهش داده و زمان دسترسی میانگین به حافظه را بهبود می‌بخشد.

کلیدواژه‌ها: حافظه نهان قربانی، نقصان برخورد، نگاشت مستقیم، تلاطم، زمان دسترسی میانگین.

فهرست مطالب

۱	مقدمه	۱
۳	مفاهیم پایه و پیش‌زمینه	۲
۳	۱-۲ سلسله مراتب حافظه	۳
۳	۲-۲ انواع نقصان در حافظه نهان	۳
۴	۱-۲-۲ نقصان اجباری	۴
۴	۲-۲-۲ نقصان ظرفیتی	۴
۵	۳-۲-۲ نقصان برخورد	۵
۶	۳ ساختار پایه و شبیه‌سازی وضعیت تلاطم	۳
۶	۱-۳ معماری پایه در شبیه‌سازی آکیتا	۶
۷	۱-۱-۳ حافظه نهان در شبیه‌سازی آکیتا	۷
۸	۲-۱-۳ حافظه اصلی در شبیه‌سازی آکیتا	۸
۹	۳-۱-۳ نحوه اتصال حافظه نهان و حافظه اصلی	۹
۱۰	۴-۱-۳ تغییرات اعمال شده در کد شبیه‌سازی	۱۰
۱۰	۲-۳ برنامه محک و ایجاد تلاطم	۱۰
۱۱	۱-۲-۳ آزمون اول	۱۱
۱۱	۲-۲-۳ آزمون دوم	۱۱

۴	طراحی و معماری حافظه نهان قربانی	۱۳
۴-۱	ویژگی‌های سخت‌افزاری، توپولوژی و ماشین حالت حافظه نهان قربانی	۱۳
۴-۱-۱	توپولوژی و نحوه اتصال پورت‌ها	۱۴
۴-۱-۲	ماشین حالت و مدیریت تراکنش‌ها	۱۵
۴-۲	مکانیزم جابه‌جایی دوطرفه، سیاست جایگزینی و مدیریت ناهمگام	۱۶
۴-۲-۱	ساختار بافر تمام‌انجمی	۱۶
۴-۲-۲	مکانیزم جابه‌جایی دوطرفه و خاصیت انحصاری	۱۶
۴-۲-۳	سیاست جایگزینی کمترین استفاده اخیر	۱۶
۴-۲-۴	مدیریت پیام‌های ناهمگام و جلوگیری از بن‌بست	۱۷
۵	ارزیابی و تحلیل نتایج شبیه‌سازی	۱۸
۵-۱	مقایسه ترافیک حافظه و تاخیر سامانه	۱۸
۵-۲	تحلیل ریاضی زمان دسترسی میانگین	۲۰
۵-۲-۱	محاسبه تئوری زمان دسترسی میانگین	۲۱
۵-۲-۲	محاسبه عملی و شبیه‌سازی شده	۲۲
۵-۲-۳	دلایل تفاوت مقادیر تئوری و عملی	۲۲
۶	نتیجه‌گیری	۲۴
	مراجع	۲۶

فصل ۱

مقدمه

شکاف فزاینده بین سرعت پردازنده‌ها و حافظه اصلی یکی از چالش‌های اساسی در معماری کامپیوتر است. برای پر کردن این شکاف از سلسله‌مراتب حافظه نهان^۱ استفاده می‌شود. با این حال محدودیت‌هایی که در ساختارهای نگاشت حافظه وجود دارد، چالش‌های جدیدی را به وجود می‌آورد. به عنوان مثال، به دلیل سادگی و سرعت بالا در پیاده‌سازی، استفاده از حافظه‌های نهان با نگاشت مستقیم^۲ یا انجمنی با درجه پایین بسیار رایج است اما این روش باعث بروز نقصان‌های برخورد^۳ می‌شود. در این حالت ممکن است چندین آدرس پرکاربرد دقیقاً به یک خط از حافظه نهان اختصاص یابند. حتی اگر ظرفیت کلی حافظه برای نگهداری داده‌ها کافی باشد، این تداخل باعث می‌شود داده‌ها پیوسته از حافظه اخراج^۴ شوند.

این رفتار ناکارآمد که به عنوان پدیده تلاطم^۵ شناخته می‌شود، سامانه را مجبور می‌کند تا داده‌ها را بارها از سطوح پایین‌تر واکشی کند. این فرایند نه تنها ترافیک بیهوده‌ای در شبکه ارتباطی حافظه ایجاد می‌کند، بلکه تاخیر سنگینی را به پردازنده تحمیل می‌نماید. برای حل این مشکل، ایده اضافه کردن یک بافر بسیار کوچک و تمام‌انجمنی^۶ به عنوان حافظه نهان قربانی^۷ مطرح می‌شود که داده‌های اخراجی از سطح اول را پیش از انتقال به سطح پایین‌تر در خود نگه می‌دارد.

در این پروژه روند ناکارآمد ذکر شده ابتدا در قالب یک معماری پایه شبیه‌سازی و مدل‌سازی شده است تا میزان هدررفت عملکرد سامانه اندازه‌گیری شود. سپس با طراحی یک پیمانۀ جدید و تغییر منطق مسیریابی

Cache^۱

Direct-Mapped^۲

Conflict Misses^۳

Evict^۴

Thrashing^۵

Fully Associative^۶

Victim Cache^۷

پیام‌ها^۸، سامانه بهینه شده است. هدف نهایی این گزارش تحلیل تاثیر این معماری جدید بر موازنه میان هزینه ساخت‌افزایی و زمان دسترسی میانگین^۹ برنامه‌ها و همچنین بررسی روند جلوگیری از بن‌بست^{۱۰} در تبادل هم‌زمان پیام‌ها است.

فصل ۲

مفاهیم پایه و پیش زمینه

در این فصل مفاهیم پایه‌ای مرتبط با ساختار حافظه نهان^۱ و چالش‌های عملکردی آن بررسی می‌شود. این مفاهیم برای درک بهتر اهداف این پروژه از جمله تشخیص انواع نقصان‌ها و طراحی معماری حافظه نهان قربانی^۲ ضروری هستند.

۱-۲ سلسله مراتب حافظه

در معماری‌های پایه و مرسوم، همان‌طور که در شکل ۱-۲ قابل مشاهده است، زمانی که پردازنده به داده‌ای نیاز دارد ابتدا آن را در حافظه نهان سطح اول جستجو می‌کند. اگر داده به دلیل نقصان در این سطح یافت نشود، از سطوح پایین‌تر واکشی می‌شود. هنگام ورود یک بلاک جدید به حافظه نهان سطح اول، ممکن است بلاک قدیمی موجود در آن جایگاه بلافاصله اخراج^۳ شده و به سطوح پایین‌تر فرستاده شود. در صورتی که پردازنده دوباره به همان بلاک اخراج شده نیاز پیدا کند، سامانه باید تاخیر سنگین خواندن دوباره آن از سطوح پایین‌تر را تحمل کند. [۱]

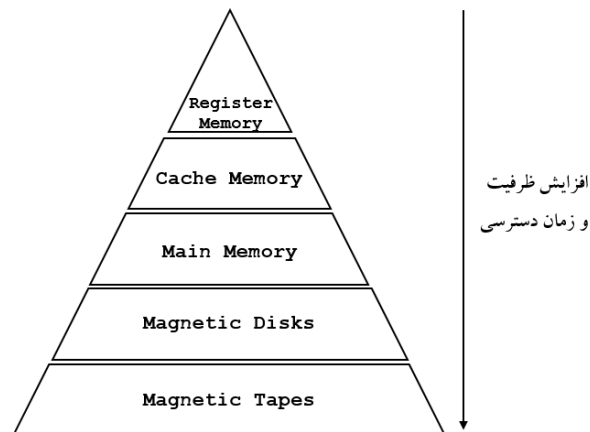
۲-۲ انواع نقصان در حافظه نهان

هنگام دسترسی پردازنده به حافظه، در صورتی که داده مورد نظر در سطح فعلی موجود نباشد، پدیده‌ای به نام نقصان اتفاق می‌افتد. نقصان‌ها به طور کلی به سه دسته اصلی تقسیم می‌شوند که درک تفاوت میان

^۱ Cache Memory

^۲ Victim Cache

^۳ Evict



شکل ۱-۲: سلسله مراتب حافظه.

آن‌ها برای تحلیل عملکرد سامانه اهمیت بالایی دارد. [۲، ۳]

۱-۲-۲ نقصان اجباری

نقصان اجباری^۴ زمانی رخ می‌دهد که پردازنده برای اولین بار به یک آدرس از حافظه دسترسی پیدا می‌کند. از آنجا که داده‌ی مربوط به این آدرس پیش از این خوانده نشده است، وقوع این نقصان و واکنشی آن از سطوح پایین‌تر اجتناب‌ناپذیر است.

۲-۲-۲ نقصان ظرفیتی

نقصان ظرفیتی^۵ زمانی اتفاق می‌افتد که گنجایش کلی حافظه‌ی نهان برای نگهداری تمام داده‌های مورد نیاز برنامه کافی نباشد. در پیاده‌سازی این پروژه و در بخش ایجاد کننده درخواست‌ها، این پدیده با واکنشی آدرس‌هایی خارج از گستره‌ی ظرفیت حافظه‌ی نهان به صورت عملی مدل‌سازی شده است.

^۴ Compulsory Miss
^۵ Capacity Miss

۳-۲-۲ نقصان برخورد

نقصان برخورد^۶ به طور ویژه در حافظه‌های نهانی رخ می‌دهد که با روش نگاشت مستقیم^۷ یا انجمنی^۸ با درجه پایین کار می‌کنند. به دلیل محدودیت‌هایی که در ساختارهای نگاشت حافظه وجود دارد، ممکن است چندین آدرس پرکاربرد به یک خط مشخص از حافظه نهان اختصاص یابند. حتی اگر ظرفیت کلی حافظه نهان برای نگهداری داده‌ها کافی باشد، تداخل آدرس‌ها باعث می‌شود داده‌ها پیوسته از حافظه اخراج شده و پدیده‌ای به نام تلاطم^۹ رخ دهد. این اتفاق سامانه را مجبور می‌کند تا داده‌ها را بارها از سطوح پایین‌تر حافظه واکشی کند که جریمه زمانی و تاخیر قابل توجهی را به همراه دارد. در کدهای شبیه‌سازی این پروژه، این رفتار ناکارآمد با ایجاد حلقه‌ای از درخواست‌های مکرر بر روی آدرس‌های متعارض پیاده‌سازی شده است.

Conflict Miss^۶
Direct-Mapped^۷
Associative^۸
Thrashing^۹

فصل ۳

ساختار پایه و شبیه‌سازی وضعیت تلاطم

در این فصل به بررسی ساختار پایه شبیه‌ساز و نحوه مدل‌سازی رفتار پردازنده برای ایجاد ترافیک حافظه می‌پردازیم. پیش از اضافه کردن هرگونه راهکار بهبود عملکرد، نیاز است تا معماری پایه به درستی مستقر شده و پدیده تلاطم^۱ که منجر به افت شدید عملکرد سامانه می‌شود به صورت دقیق شبیه‌سازی و اندازه‌گیری شود. برای این منظور از تولید الگوهای دسترسی متناوب به حافظه استفاده شده است.

۳-۱ معماری پایه در شبیه‌ساز آکیتا

شبیه‌ساز آکیتا^۲ یک چارچوب شبیه‌سازی رویدادمحور^۳ در معماری کامپیوتر است که به زبان گو^۴ توسعه یافته است. این شبیه‌ساز از قطعات سخت‌افزاری مستقلی تشکیل شده است که هر کدام دارای پورت‌های ارتباطی مشخصی هستند و با ارسال پیام با یکدیگر ارتباط برقرار می‌کنند. آکیتا به طراحان اجازه می‌دهد تا زمان‌بندی دقیق چرخه‌های ساعت، تأخیر در صف‌های ارسال پیام و عملکرد قطعات حافظه را با دقت بالایی مدل‌سازی کنند. در این پروژه، از آکیتا برای ساخت یک سامانه پایه متشکل از یک پردازنده تولیدکننده الگو، یک سطح از حافظه نهان و یک کنترل‌کننده حافظه اصلی استفاده شده است. [۴]

^۱Thrashing

^۲Akita

^۳Event-driven

^۴Go

۱-۱-۳ حافظه نهان در شبیه‌ساز آکیتا

پیکربندی حافظه نهان سطح یک (L1) در معماری پایه در فایل main.go و با استفاده از سازنده writeback پیاده‌سازی شده است. مشخصات و ویژگی‌های عملکردی این حافظه به شرح زیر است:

- **ظرفیت و اندازه بلاک:** ظرفیت کل این حافظه برابر یک کیلوبایت (1 KB) در نظر گرفته شده است. با توجه به تنظیم مقدار لگاریتم اندازه بلاک برابر ۶ ($\text{Log2BlockSize} = 6$)، اندازه هر بلاک داده در این حافظه ۶۴ بایت است. این یعنی حافظه نهان در مجموع دارای ۱۶ بلاک فیزیکی است. ظرفیت و اندازه حافظه نهان در شکل ۱-۳ نمایش داده شده است.

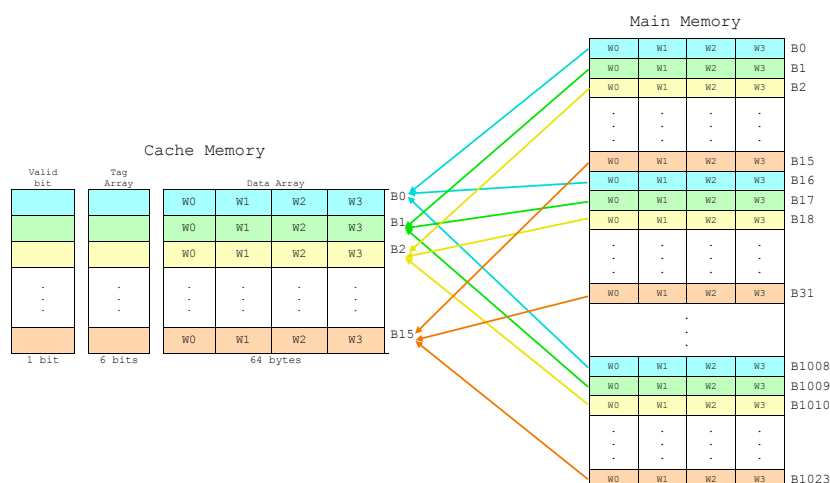
Valid bit	Tag Array	Data Array			
		W0	W1	W2	W3
		W0	W1	W2	W3
		W0	W1	W2	W3
⋮	⋮	⋮	⋮	⋮	⋮
		W0	W1	W2	W3
1 bit	6 bits	64 bytes			

شکل ۱-۳: ساختار و ظرفیت حافظه نهان.

- **سیاست نگاشت:** درجه انجمنی^۵ این حافظه برابر ۱ تنظیم شده است. این مقدار بیانگر استفاده از روش نگاشت مستقیم^۶ است. در این روش، هر آدرس حافظه دقیقاً به یک خط مشخص در حافظه نهان نگاشت می‌شود که زمینه را برای بروز نقصان‌های برخورد^۷ فراهم می‌کند. سیاست نگاشت به صورت تصویری در شکل ۲-۳ آمده است.

- **تاخیر دسترسی:** تاخیر اصابت^۸ برابر ۱ چرخه ساعت است که به صورت مساوی (نیم چرخه) بین تاخیر دایرکتوری و تاخیر بانک‌های حافظه تقسیم شده است.

^۵ Associativity
^۶ Direct-Mapped
^۷ Conflict Miss
^۸ Hit Time



شکل ۳-۲: سیاست نگاشت مستقیم.

- مدیریت درخواست‌های معلق: این ماژول دارای ۴ مدخل در ثبات نگهداری وضعیت نقصان^۹ است. این ویژگی به حافظه نهان اجازه می‌دهد در صورت وقوع نقصان، تا ۴ درخواست را به صورت هم‌زمان به سطوح پایین‌تر ارسال کرده و منتظر پاسخ بماند بدون این که متوقف شود.

علاوه بر موارد فوق، این ماژول از سیاست بازنویسی^{۱۰} استفاده می‌کند، به این معنا که تغییرات داده تا زمانی که خط مورد نظر از حافظه اخراج نشود، به حافظه اصلی فرستاده نخواهد شد.

۳-۱-۲ حافظه اصلی در شبیه‌ساز آکیتا

برای مدل‌سازی حافظه اصلی در حالت پایه، از یک کنترل‌کننده حافظه ایده‌آل^{۱۱} استفاده شده است. این کنترل‌کننده پیچیدگی‌های داخلی ماژول‌های رم^{۱۲} (مانند تازه کردن ردیف‌ها و تاخیرهای متغیر بانک‌ها) را نادیده گرفته و به جای آن یک مدل ساده و قابل پیش‌بینی ارائه می‌دهد. ظرفیت این حافظه ۶۴ کیلوبایت تعیین شده است که برای برنامه‌های محک^{۱۳} این پروژه کاملاً پاسخگو است. مهم‌ترین مشخصه این بخش، تاخیر بسیار بالای آن است که برابر ۱۰۰ چرخه ساعت ($MEM_LATENCY = 100$) تنظیم شده است. این تاخیر سنگین باعث می‌شود که هر بار داده‌ای در حافظه نهان یافت نشود، پردازنده برای دریافت آن جریمه زمانی بسیار بزرگی را متحمل شود. ساختار حافظه اصلی در شکل ۳-۳ نمایش داده شده است.

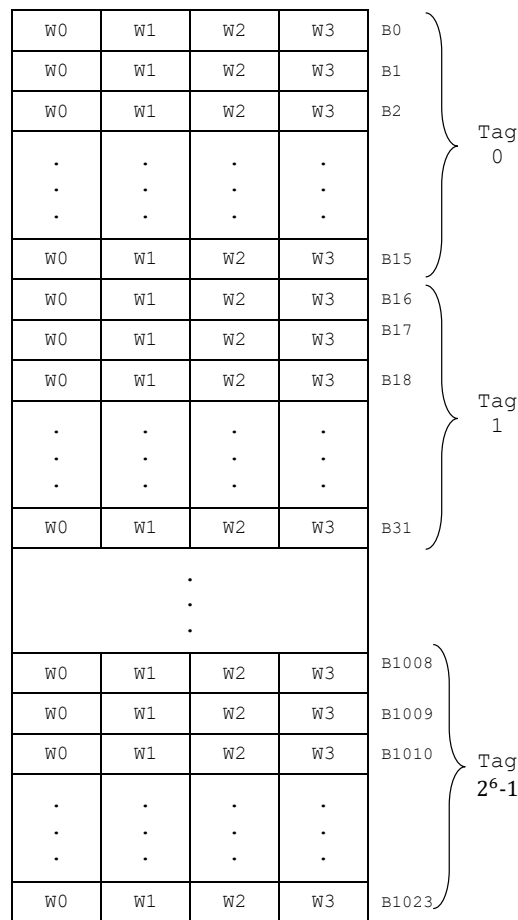
Miss Status Holding Register - MSHR^۹

Write-back^{۱۰}

Ideal Memory Controller^{۱۱}

RAM^{۱۲}

Benchmark^{۱۳}



شکل ۳-۳: ساختار و ظرفیت حافظه اصلی.

۳-۱-۳ نحوه اتصال حافظه نهان و حافظه اصلی

در شبیه‌ساز آکیتا، قطعات سخت‌افزاری از طریق سامانه اتصال مستقیم^{۱۴} به یکدیگر متصل می‌شوند. هر ماژول دارای پورت‌های بالا^{۱۵} (برای دریافت درخواست از قطعات نزدیک‌تر به پردازنده) و پایین^{۱۶} (برای ارسال درخواست به سطوح دورتر حافظه) است. در معماری پایه، پورت حافظه (Mem) در پردازنده به پورت بالای حافظه نهان متصل می‌شود. سپس پورت پایین حافظه نهان مستقیماً به پورت بالای کنترل‌کننده حافظه اصلی وصل می‌گردد. همچنین برای هدایت صحیح بسته‌های داده، از یک نگاشت‌گر پورت یگانه^{۱۷} استفاده شده است تا تمامی درخواست‌های خروجی از حافظه نهان را بدون نیاز به مسیریابی پیچیده،

^{۱۴} Direct Connection

^{۱۵} Top Port

^{۱۶} Bottom Port

^{۱۷} Single Port Mapper

مستقیماً به سمت حافظه اصلی هدایت کند. تمامی این اتصالات در فرکانس ۱ گیگاهرتز همگام‌سازی شده‌اند.

۳-۱-۴ تغییرات اعمال‌شده در کد شبیه‌ساز

برای ارزیابی عملکرد حافظه نهان، قابلیت جمع‌آوری آمار به ماژول Write-Back Cache در شبیه‌ساز آکیتا اضافه شد. بدین منظور، در فایل `mem/cache/writeback/writebackcache.go`، شمارنده‌هایی برای ثبت تعداد اصابت خواندن^{۱۸}، نقصان خواندن^{۱۹}، اصابت نوشتن^{۲۰}، نقصان نوشتن^{۲۱}، اخراجی‌ها^{۲۲} و بازنویسی‌ها به ساختار حافظه نهان افزوده شد. سپس در فایل‌های مربوط به پردازش درخواست‌ها و جایگزینی بلوک‌ها، این شمارنده‌ها در محل مناسب به‌روزرسانی شدند تا آمار هر دسترسی به‌صورت دقیق ثبت شود. علاوه بر این، برای پیاده‌سازی حافظه نهان قربانی نیز در فایل `victimcache.go` شمارنده‌های مربوط به اصابت‌های حافظه نهان قربانی^{۲۳}، نقصان‌های حافظه نهان قربانی^{۲۴} و ترافیک حافظه^{۲۵} اضافه شد تا عملکرد این سطح از حافظه نیز قابل ارزیابی باشد. در نهایت، در فایل `tracecpu.go` واقع در پروژه اصلی، این آمارها پس از اجرای برنامه محک نمایش داده شده و با استفاده از آن‌ها نرخ Hit/Miss و همچنین Average Memory Access Time (AMAT) برای هر دو حالت، با و بدون حافظه نهان قربانی، محاسبه و گزارش می‌شود.

۳-۲ برنامه محک و ایجاد تلاطم

برای سنجش عملکرد حافظه نهان و ارزیابی راه‌حل‌های پیشنهادی، نیاز به تولید ترافیک حافظه داریم. ماژول TraceCPU در این پروژه وظیفه دارد به جای اجرای یک کد اسمبلی واقعی، مستقیماً درخواست‌های خواندن و نوشتن را به آدرس‌های از پیش تعیین شده ارسال کند. در این بخش دو الگوی متفاوت دسترسی به حافظه به نام‌های الگوی کوچک (small trace) و الگوی بزرگ (large trace) طراحی شده است که هدف اصلی آن‌ها ایجاد ترافیک متناوب و شبیه‌سازی بدترین حالت برای حافظه نهان نگاشت مستقیم، یعنی پدیده تلاطم است.

^{۱۸}Read Hit

^{۱۹}Read Miss

^{۲۰}Write Hit

^{۲۱}Write Miss

^{۲۲}Evictions

^{۲۳}Victim Cache Hits

^{۲۴}Victim Cache Misses

^{۲۵}Memory Traffic

۱-۲-۳ آزمون اول

آزمون اول با استفاده از الگوی smallTrace اجرا می‌شود و هدف آن بررسی رفتار حافظه با تعداد کمی درخواست، اما با نرخ برخورد بالا است. این الگو به سه فاز اصلی تقسیم می‌شود:

۱. فاز پر کردن حافظه: در ابتدا ۱۶ درخواست نوشتن متوالی با فاصله ۶۴ بایت (از آدرس 0x0000 تا 0x03C0) به حافظه ارسال می‌شود. چون ظرفیت حافظه نهان دقیقا ۱۶ بلاک ۶۴ بایتی است، این فاز کل ظرفیت حافظه نهان را پر می‌کند و به طور طبیعی ۱۶ نقصان اولیه^{۲۶} ایجاد می‌کند.

۲. فاز تایید جایگذاری: سپس پردازنده ۶ بلاک اول را دوباره می‌خواند که تمامی این خواندن‌ها منجر به اصابت موفقیت‌آمیز در حافظه نهان می‌شوند.

۳. فاز ایجاد تلاطم: در این مرحله پردازنده درخواست نوشتن در آدرس 0x0400 را ارسال می‌کند. فاصله آدرس 0x0400 (معادل ۱۰۲۴ بایت) با آدرس 0x0000 دقیقا برابر با ظرفیت کل حافظه نهان (۱ کیلوبایت) است. به دلیل ساختار نگاشت مستقیم، هر دو آدرس به یک خط فیزیکی در حافظه نهان ختم می‌شوند. با واکشی 0x0400، داده 0x0000 اخراج می‌شود. بلافاصله در دستور بعدی، پردازنده دوباره آدرس 0x0000 را درخواست می‌کند که باعث اخراج 0x0400 می‌گردد. این فرایند که داده‌ها پیش از استفاده مجدد از حافظه اخراج می‌شوند، پدیده تلاطم نام دارد. این الگو برای بلاک‌های بعدی نیز تکرار می‌شود تا ترافیک بهبود یافته زیادی روانه حافظه اصلی گردد.

۲-۲-۳ آزمون دوم

آزمون دوم که با متغیر largeTrace شناخته می‌شود، برای شبیه‌سازی یک بار کاری سنگین و طولانی‌تر طراحی شده است تا تاثیر تاخیر حافظه بر زمان کل اجرا را به شکل واضح‌تری نشان دهد. در این آزمون، پردازنده پس از پر کردن اولیه حافظه، وارد یک حلقه تکرار ۳۰ تایی می‌شود. درون این حلقه اصلی، حلقه‌های داخلی دیگری وجود دارند که ۱۰ بار به طور متناوب بین آدرس‌های کاملا متداخل درخواست ارسال می‌کنند (برای مثال تناوب مداوم بین آدرس‌های 0x0000 و 0x0400، و سپس 0x0440 و 0x0040). این حجم از درخواست‌های متناوب به آدرس‌هایی که با یکدیگر برخورد دارند، باعث می‌شود نرخ اصابت در حافظه نهان سطح یک به شدت سقوط کند و سامانه در یک چرخه باطل از اخراج و واکشی مجدد گرفتار شود. این آزمون بیش از ۶۰۰ دسترسی به حافظه را شبیه‌سازی می‌کند و در معماری پایه، تقریبا تمام این درخواست‌ها به عنوان نقصان برخورد شناخته شده و با تحمیل تاخیر ۱۰۰ چرخه‌ای، باعث کندی شدید

^{۲۶} Cold Miss

زمان نهایی اجرای برنامه می‌شوند.

فصل ۴

طراحی و معماری حافظه نهان قربانی

در این فصل به بررسی جزئیات طراحی، ساختار سخت‌افزاری و منطق پیاده‌سازی حافظه نهان قربانی^۱ می‌پردازیم. همان‌طور که پیش‌تر اشاره شد، حافظه نهان قربانی یک بافر کوچک و تمام‌انجمنی^۲ است که بین حافظه نهان سطح یک و حافظه اصلی قرار می‌گیرد تا از افت عملکرد ناشی از نقصان‌های برخورد جلوگیری کند^۳. این پیمانیه در فایل victimcache.go پیاده‌سازی شده است که در ادامه ابعاد مختلف آن تشریح می‌شود. [۶، ۵، ۳]

۴-۱ ویژگی‌های سخت‌افزاری، توپولوژی و ماشین حالت حافظه نهان قربانی

پیکربندی و مشخصات فنی حافظه نهان قربانی در ساختار VCSpec و تابع DefaultVCSpec تعیین شده است. ویژگی‌های کلیدی این ساختار شامل فرکانس کاری ۱ گیگاهرتز، اندازه بلاک ۶۴ بایت (مطبق با اندازه بلاک در حافظه نهان سطح یک) و تعداد ۸ بلاک فیزیکی برای ذخیره‌سازی داده‌های اخراجی است. تاخیر داخلی این ماژول برابر ۵ چرخه ساعت^۴ تنظیم شده است که جریمه بررسی بافر قربانی را مدل‌سازی می‌کند.

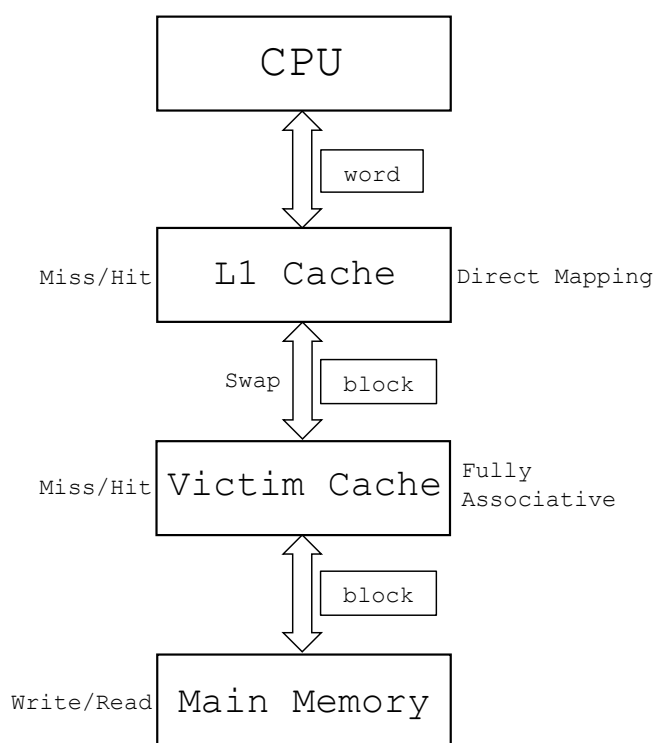
^۱Victim Cache
^۲Fully Associative
^۳Conflict Misses
^۴Latency = 5

۱-۱-۴ توپولوژی و نحوه اتصال پورت‌ها

در معماری دارای حافظه نهان قربانی، نحوه اتصال قطعات تغییر می‌کند تا ترافیک خروجی از حافظه نهان سطح یک مستقیماً به حافظه اصلی نرود. این توپولوژی در فایل main.go به کمک سامانه اتصال مستقیم پیاده‌سازی شده است:

- **پورت بالایی (TopPort):** این پورت به عنوان ورودی حافظه قربانی عمل کرده و به پورت پایینی حافظه نهان سطح یک متصل می‌شود. تمامی درخواست‌های خواندن و نوشتن که از سطح یک خارج می‌شوند (مانند نقصان‌ها و اخراج‌ها) به این پورت وارد می‌شوند.
- **پورت پایینی (BottomPort):** این پورت به عنوان خروجی حافظه قربانی به شمار می‌رود و مستقیماً به پورت بالای کنترل‌کننده حافظه اصلی متصل می‌گردد. درخواست‌هایی که در بافر قربانی نیز یافت نشوند (نقصان در قربانی) یا بلاک‌هایی که به دلیل تکمیل ظرفیت باید اخراج شوند، از طریق این پورت به حافظه اصلی ارسال می‌گردند.

توپولوژی و نحوه اتصال پورت‌ها در شکل ۱-۴ نمایش داده شده است.



شکل ۱-۴: توپولوژی و نحوه اتصال پورت‌ها.

۴-۱-۲ ماشین حالت و مدیریت تراکنش‌ها

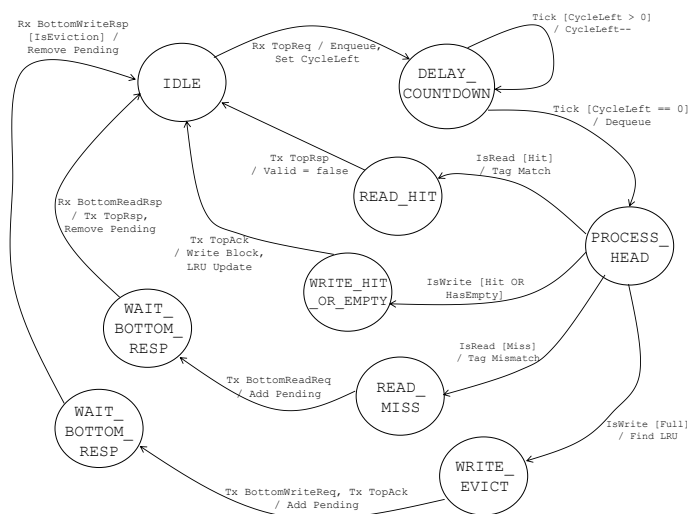
ماژول حافظه نهان قربانی به عنوان یک کامپوننت زمان‌دار^۵ در هر چرخه ساعت تابع Tick را اجرا می‌کند. مدیریت تراکنش‌ها در این ماژول از طریق یک ماشین حالت ناهمگام انجام می‌شود که چرخه حیات هر درخواست را کنترل می‌کند:

۱. دریافت درخواست: پیام‌های ورودی از طریق TopPort دریافت شده و در صف تراکنش‌های بالا^۶ قرار می‌گیرند.

۲. مدیریت تاخیر: تابع countDown در هر چرخه ساعت، مقدار شمارنده تاخیر باقی‌مانده^۷ را برای تراکنش‌های در صف کاهش می‌دهد تا تاخیر سخت‌افزاری ۵ چرخه‌ای اعمال شود.

۳. پردازش سرصف: پس از اتمام تاخیر، تابع processHeadTransaction تراکنش را بر اساس نوع آن (خواندن یا نوشتن) به توابع تخصصی هدایت می‌کند تا اصابت یا نقصان بررسی شود.

ماشین حالت حافظه نهان قربانی در شکل ۴-۲ رسم شده است.



شکل ۴-۲: ماشین حالت حافظه نهان قربانی.

^۵Ticking Component
^۶vcTopTransaction
^۷CycleLeft

۲-۴ مکانیزم جابه‌جایی دوطرفه، سیاست جایگزینی و مدیریت ناهمگام

منطق اصلی حافظه نهان قربانی در فایل victimcache.go و از طریق توابع پردازش خواندن، نوشتن و پاسخ‌های حافظه پیاده‌سازی شده است که در ادامه به تفکیک بررسی می‌شوند.

۱-۲-۴ ساختار بافر تمام‌انجمی

برخلاف حافظه نهان سطح یک که از نگاشت مستقیم استفاده می‌کند، حافظه نهان قربانی به صورت یک آرایه از ساختارهای VCBLOCK پیاده‌سازی شده است که هر مدخل شامل برچسب^۸، بیت اعتبار^۹، آرایه داده^{۱۰} و زمان آخرین دسترسی است. به دلیل عدم استفاده از تابع نگاشت ریاضی، هنگام جستجوی یک آدرس، کد با استفاده از یک حلقه تمام خانه‌های آرایه را بررسی می‌کند تا اصابت یا نقصان را تشخیص دهد که این دقیقاً رفتار سخت‌افزاری یک بافر تمام‌انجمی را بازسازی می‌کند.

۲-۲-۴ مکانیزم جابه‌جایی دوطرفه و خاصیت انحصاری

هنگامی که پردازنده در حافظه نهان سطح یک دچار نقصان می‌شود اما داده مورد نظر در حافظه نهان قربانی موجود است (اصابت در قربانی)، فرایند جابه‌جایی دوطرفه^{۱۱} رخ می‌دهد. این فرایند در تابع handleRead پیاده‌سازی شده است: داده درخواستی به سرعت به سمت حافظه نهان سطح یک بازگردانده می‌شود. برای حفظ خاصیت انحصاری^{۱۲} و جلوگیری از وجود نسخه تکراری از داده در سطح یک و قربانی، بلافاصله پس از اصابت، بیت اعتبار آن مدخل در قربانی غیرمعتبر می‌گردد (Valid = false). این کار باعث می‌شود جایگاه مورد نظر در قربانی آزاد شود تا بلاک اخراجی بعدی بتواند در آن جای گیرد.

۳-۲-۴ سیاست جایگزینی کمترین استفاده اخیر

هنگامی که حافظه نهان سطح یک خطی را اخراج می‌کند و آن را به حافظه قربانی می‌فرستد (handleWrite)، اگر بافر قربانی کاملاً پر باشد، نیاز به یک سیاست جایگزینی است. در این کد از الگوریتم کمترین استفاده

^۸Tag

^۹Valid

^{۱۰}Data

^{۱۱}Swap

^{۱۲}Exclusive Property

اخیر^{۱۳} استفاده شده است: کد با بررسی فیلد LastAccessTime در تمامی بلاک‌های معتبر، قدیمی‌ترین بلاک را شناسایی می‌کند. سپس آن بلاک قدیمی را از طریق BottomPort به عنوان یک درخواست نوشتن به حافظه اصلی منتقل می‌کند تا داده‌های تغییر یافته از دست نروند. سپس داده جدید جایگزین آن بلاک می‌شود و زمان دسترسی آن به روز می‌گردد.

۴-۲-۴ مدیریت پیام‌های ناهمگام و جلوگیری از بن‌بست

به دلیل تاخیر بالای حافظه اصلی، سامانه نباید متوقف شود. مدیریت ناهمگام پاسخ‌ها و جلوگیری از بروز بن‌بست^{۱۴} از طریق صف vcPendingBottom انجام می‌شود: وقتی درخواستی در قربانی یافت نمی‌شود (نقصان در قربانی)، یا وقتی یک بلاک به ناچار به سمت حافظه اصلی اخراج می‌شود، درخواست مربوطه همراه با شناسه‌های اصلی آن در صف pendingBottom ذخیره می‌گردد و کنترل به شبیه‌ساز بازمی‌گردد. هنگامی که حافظه اصلی پس از گذشت چرخه‌های فراوان پاسخ خود را برمی‌گرداند، توابع completeBottomRead و completeBottomWrite پیام برگشتی را دریافت کرده، با استفاده از شناسه‌ها مرجع اولیه آن را پیدا می‌کنند و پاسخ را به مقصد درست در بالا هدایت می‌نمایند. در مورد اخراج‌های پس‌زمینه^{۱۵} نیز با تنظیم پرچم مربوطه مشخص می‌شود که نیازی به ارسال پاسخ به سمت بالا نیست، که این رویکرد از ایجاد حالت‌های معلق و بن‌بست در شبکه جلوگیری می‌کند.

^{۱۳} Least Recently Used - LRU

^{۱۴} Deadlock

^{۱۵} Eviction

فصل ۵

ارزیابی و تحلیل نتایج شبیه‌سازی

در این فصل به بررسی خروجی‌های به دست آمده از اجرای شبیه‌ساز آکیتا می‌پردازیم. هدف اصلی این بخش، نشان دادن تاثیر چشمگیر اضافه شدن حافظه نهان قربانی^۱ بر کاهش ترافیک حافظه اصلی و بهبود زمان اجرای برنامه‌ها در شرایط بروز تلاطم است. برای این منظور، نتایج دو برنامه محک با الگوهای دسترسی متفاوت در دو معماری پایه و معماری بهبودیافته مقایسه می‌شوند.

۵-۱ مقایسه ترافیک حافظه و تاخیر سامانه

نتایج خروجی شبیه‌ساز به وضوح نشان می‌دهد که استفاده از یک بافر کوچک تمام‌انجمنی تا چه اندازه می‌تواند از هدررفت منابع سامانه جلوگیری کند. ما این موضوع را در دو مقیاس متفاوت بررسی می‌کنیم:

در آزمون اول که شامل ۴۰ درخواست دسترسی به حافظه (۲۴ نوشتن و ۱۶ خواندن) است، نرخ اصابت^۲ در حافظه نهان سطح یک برابر با ۳۰ درصد است. در معماری پایه، ۲۸ درخواست به دلیل بروز نقصان به حافظه اصلی ارسال می‌شود که زمان اجرای کل را به ۳۱۴۶ چرخه ساعت می‌رساند. با اضافه شدن حافظه نهان قربانی، ۶ مورد از این نقصان‌ها در بافر قربانی اصابت می‌کنند (نرخ اصابت ۲۱/۴۳ درصد در قربانی). این امر ترافیک ارسالی به حافظه اصلی را به ۲۲ درخواست کاهش داده و باعث می‌شود زمان اجرای کل برنامه به ۲۷۳۰ چرخه تقلیل یابد. خروجی شبیه‌ساز برای این آزمون در شکل‌های ۵-۱ و ۵-۲ آمده است.

تفاوت اصلی و اوج کارایی حافظه قربانی در آزمون دوم آشکار می‌شود. در این آزمون که برای شبیه‌سازی

Victim Cache^۱
Hit Rate^۲

```

=====
Test 1
=====
Writes      : 24
Reads       : 16

L1 Cache
Read Hits   : 10
Read Misses : 6
Write Hits   : 2
Write Misses : 22
WriteBacks   : 11
Hit Rate     : 30.00%

Victim Cache
Hits        : N/A
Misses      : N/A
Hit Rate     : N/A

Memory Traffic : 28 requests
Theoretical AMAT : 71.00 cycles
Simulated AMAT  : 78.63 cycles
Execution Time  : 3146 cycles (0.000003146 s)

```

شکل ۵-۱: خروجی شبیه‌ساز برای آزمون اول: معماری پایه.

```

=====
Test 1 (with Victim Cache)
=====
Writes      : 24
Reads       : 16

L1 Cache
Read Hits   : 10
Read Misses : 6
Write Hits   : 2
Write Misses : 22
WriteBacks   : 11
Hit Rate     : 30.00%

Victim Cache
Hits        : 6
Misses      : 22
Hit Rate     : 21.43%

Memory Traffic : 22 requests
Theoretical AMAT : 56.75 cycles
Simulated AMAT  : 68.22 cycles
Execution Time  : 2730 cycles (0.000002730 s)

```

شکل ۵-۲: خروجی شبیه‌ساز برای آزمون اول: معماری دارای حافظه قربانی.

تلاطم شدید طراحی شده است، پردازنده ۲۴۷۶ درخواست (۱۲۴۶ نوشتن و ۱۲۳۰ خواندن) ارسال می‌کند. به دلیل متداخل بودن آدرس‌ها، نرخ اصابت در حافظه نهان سطح یک به عدد فاجعه‌بار ۲/۳۸ درصد سقوط می‌کند. در معماری پایه، این تلاطم باعث می‌شود ۲۴۱۷ درخواست روانه حافظه اصلی شود و زمان اجرای برنامه به ۲۶۷۴۵۱ چرخه ساعت برسد. اما با فعال‌سازی حافظه نهان قربانی، شاهد یک بهبود خیره‌کننده هستیم. حافظه قربانی موفق می‌شود ۱۲۳۰ مورد از داده‌های اخراجی را در خود نگه دارد و نرخ اصابت فوق‌العاده ۵۰/۸۹ درصد را ثبت کند. این کار ترافیک ارسالی به حافظه اصلی را به نصف کاهش داده (۱۱۸۷ درخواست) و زمان اجرای نهایی برنامه را با بیش از ۴۰ درصد بهبود، به ۱۵۸۹۱۰ چرخه می‌رساند. این اعداد به طور قطعی ثابت می‌کنند که حافظه قربانی راهکاری بسیار موثر برای رفع نقصان‌های برخورد است. خروجی شبیه‌ساز برای آزمون دوم در شکل‌های ۵-۳ و ۵-۴ آمده است.

```

=====
Test 2
=====
Writes      : 1246
Reads       : 1230

L1 Cache
Read Hits   : 0
Read Misses : 1230
Write Hits   : 59
Write Misses : 1187
WriteBacks   : 1234
Hit Rate     : 2.38%

Victim Cache
Hits        : N/A
Misses       : N/A
Hit Rate     : N/A

Memory Traffic : 2417 requests
Theoretical AMAT : 98.62 cycles
Simulated AMAT  : 108.02 cycles
Execution Time  : 267451 cycles (0.000267451 s)
=====

```

شکل ۵-۳: خروجی شبیه‌ساز برای آزمون دوم: معماری پایه.

```

=====
Test 2 (with Victim Cache)
=====
Writes      : 1246
Reads       : 1230

L1 Cache
Read Hits   : 0
Read Misses : 1230
Write Hits   : 59
Write Misses : 1187
WriteBacks   : 1234
Hit Rate     : 2.38%

Victim Cache
Hits        : 1230
Misses       : 1187
Hit Rate     : 50.89%

Memory Traffic : 1187 requests
Theoretical AMAT : 51.42 cycles
Simulated AMAT  : 64.18 cycles
Execution Time  : 158910 cycles (0.000158910 s)
=====

```

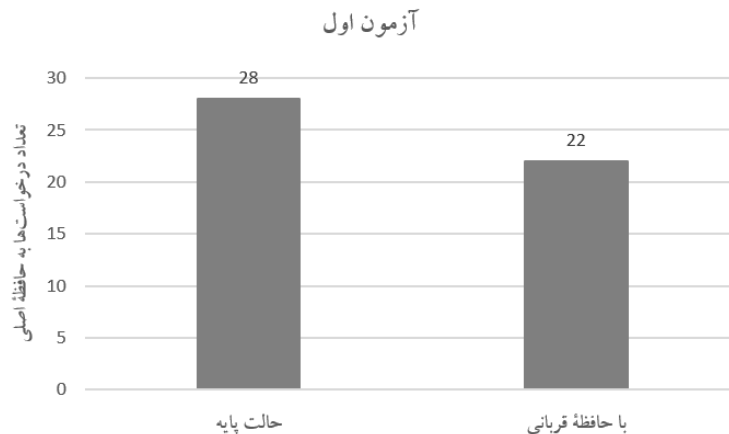
شکل ۵-۴: خروجی شبیه‌ساز برای آزمون دوم: معماری دارای حافظه قربانی.

ضمناً در نمودارهای ۵-۵ و ۶-۵ مقایسه میزان کاهش ترافیک حافظه اصلی نشان داده شده است.

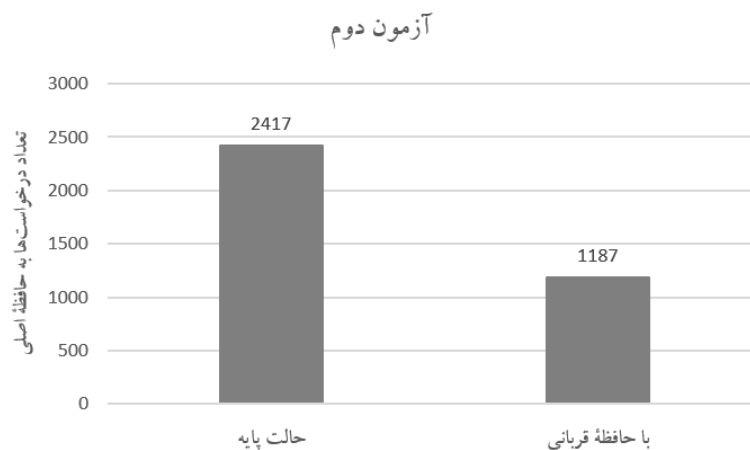
۲-۵ تحلیل ریاضی زمان دسترسی میانگین

زمان دسترسی میانگین^۳ یکی از مهم‌ترین معیارهای ارزیابی عملکرد سامانه‌های حافظه است. در شبیه‌ساز ما، این معیار به دو روش تئوری و عملی محاسبه شده است که در ادامه به تشریح آن‌ها می‌پردازیم.

Average Memory Access Time - AMAT^۳



شکل ۵-۵: مقایسه میزان ترافیک ارسالی به حافظه اصلی قبل و بعد از اضافه شدن حافظه نهان قربانی: آزمون اول.



شکل ۵-۶: مقایسه میزان ترافیک ارسالی به حافظه اصلی قبل و بعد از اضافه شدن حافظه نهان قربانی: آزمون دوم.

۱-۲-۵ محاسبه تئوری زمان دسترسی میانگین

محاسبه تئوری بر اساس فرمول‌های استاندارد معماری کامپیوتر و صرفاً با استفاده از نرخ اصابت و تاخیرهای ثابت قطعات انجام می‌شود. در فایل `tracecpu.go`، این فرمول برای معماری دارای حافظه قربانی به شکل زیر پیاده‌سازی شده است [۷]:

$$AMAT = HitTime_{L_1} + MissRate_{L_1} \times (HitRate_{VC} \times Penalty_{VC} + MissRate_{VC} \times Penalty_{L_2})$$

برای مثال در آزمون دوم (با حافظه قربانی)، تاخیر سطح یک برابر ۱، نرخ نقصان سطح یک در حدود ۰/۹۷۶، نرخ اصابت قربانی ۰/۵۰۸۹، تاخیر قربانی ۵ و تاخیر حافظه اصلی ۱۰۰ چرخه است. با جایگذاری

این اعداد، شبیه‌ساز عدد ۵۱/۴۲ چرخه را به عنوان زمان دسترسی میانگین تئوری محاسبه می‌کند. در حالی که همین فرمول برای معماری پایه در آزمون دوم عدد ۹۸/۶۲ چرخه را نشان می‌دهد. این کاهش شدید در فرمول ریاضی، نشان‌دهنده تأثیر وزن بالای نرخ اصابت قربانی در مهار جریمه سنگین ۱۰۰ چرخه‌ای حافظه اصلی است.

۵-۲-۲ محاسبه عملی و شبیه‌سازی شده

شبیه‌ساز آکیتا امکان محاسبه زمان دسترسی میانگین را به صورت کاملاً عملی و بر اساس زمان‌بندی واقعی رویدادها فراهم کرده است. در این کد، هر زمان که پردازنده درخواستی ارسال می‌کند، شناسه آن پیام به همراه زمان دقیق ارسال در یک ساختار داده به نام reqStartTimes ذخیره می‌شود. هنگامی که تابع processResponses پاسخ نهایی را از حافظه دریافت می‌کند، زمان فعلی سامانه را از زمان شروع درخواست کم کرده و تاخیر واقعی آن دسترسی مشخص را به دست می‌آورد. سپس مجموع این تاخیرها (totalAccessLatency) بر تعداد کل درخواست‌ها تقسیم می‌شود. خروجی شبیه‌ساز نشان می‌دهد که مقدار عملی زمان دسترسی میانگین در آزمون دوم برای حالت پایه برابر ۱۰۸/۰۲ چرخه و برای حالت دارای حافظه قربانی برابر ۶۴/۱۸ چرخه است.

۵-۲-۳ دلایل تفاوت مقادیر تئوری و عملی

همان‌طور که در نتایج مشاهده می‌شود، مقادیر عملی همواره مقداری بیشتر از مقادیر محاسبه شده در فرمول تئوری هستند (برای مثال در آزمون دوم، مقدار تئوری ۵۱/۴۲ و مقدار عملی ۶۴/۱۸ چرخه است). این اختلاف ناشی از محدودیت‌ها و واقعیت‌های سخت‌افزاری است که در فرمول ریاضی نادیده گرفته می‌شوند. دلایل اصلی این تفاوت عبارتند از:

- **ترافیک بازنویسی^۴:** فرمول تئوری به طور معمول رفتار مسیرهای خواندن را مدل می‌کند. اما در شبیه‌ساز، ما شاهد ۱۲۳۴ عملیات بازنویسی در آزمون دوم هستیم. اخراج این بلاک‌های کثیف^۵ نیازمند ارسال بسته‌های حجیم داده به سمت حافظه اصلی است که پهنای باند را اشغال کرده و باعث ایجاد تاخیر برای درخواست‌های بعدی می‌شود.
- **رقابت بر سر پورت‌ها و گذرگاه^۶:** در حالت تئوری فرض بر این است که هر درخواست بدون تداخل

^۴ Write-back Traffic
^۵ Dirty Blocks
^۶ Bus Contention

با درخواست‌های دیگر مسیر خود را طی می‌کند. اما در شبکه شبیه‌ساز، پیام‌ها باید در صف پورت‌های ارسال و دریافت منتظر بمانند و اگر گذرگاه اشغال باشد، سامانه دچار توقف‌های ساختاری می‌شود.

- **محدودیت ثبات درخواست‌های معلق^v:** حافظه نهان سطح یک تنها اجازه نگهداری ۴ درخواست معلق را دارد. در صورت بروز نقصان‌های متوالی، این ظرفیت به سرعت پر شده و پردازنده مجبور است تا زمان دریافت پاسخ از سطوح پایین‌تر متوقف شود که این زمان‌های انتظار در مقدار عملی بازتاب پیدا می‌کنند اما در فرمول تئوری لحاظ نمی‌گردند.

فصل ۶

نتیجه‌گیری

در پروژه حاضر، چالش افت عملکرد ناشی از نقصان‌های برخورد در حافظه‌های نهان با معماری نگاشت مستقیم مورد بررسی و ارزیابی دقیق قرار گرفت. پدیده تلاطم که در اثر رقابت آدرس‌های متداخل برای قرارگیری در یک خط فیزیکی مشترک ایجاد می‌شود، می‌تواند ترافیک سنگین و بیهوده‌ای را به سطوح پایین‌تر حافظه تحمیل کند. برای رفع این مشکل، یک پیمانه سخت‌افزاری جدید به نام حافظه نهان قربانی طراحی و در بستر شبیه‌ساز آکیتا پیاده‌سازی شد.

بررسی طراحی و معماری این ماژول نشان داد که اضافه کردن یک بافر بسیار کوچک تمام‌انجمنی (با ظرفیت تنها هشت بلاک) میان حافظه نهان سطح یک و حافظه اصلی، می‌تواند به عنوان یک لایه محافظ در برابر اخراج‌های مکرر عمل کند. پیاده‌سازی مکانیزم جابه‌جایی دوطرفه و حفظ خاصیت انحصاری داده‌ها تضمین کرد که ظرفیت محدود این بافر بهینه‌ترین استفاده را داشته باشد و از ذخیره داده‌های تکراری در سطوح مختلف جلوگیری شود. افزون بر این، با استفاده از سیاست جایگزینی کمترین استفاده اخیر، بلوک‌های قدیمی‌تر به درستی اخراج شده و در صورت نیاز به حافظه اصلی فرستاده شدند تا داده‌ای از دست نرود. یکی از دستاوردهای مهم در بخش توسعه کد، مدیریت صحیح پیام‌های ناهمگام و جلوگیری از بروز بن‌بست در شبکه ارتباطی شبیه‌ساز بود که به کمک ایجاد صف برای درخواست‌های در انتظار محقق گردید.

نتایج به دست آمده از اجرای شبیه‌ساز بر روی برنامه‌های محک به روشنی اثربخشی این معماری را ثابت کرد. در آزمون‌های طراحی شده که تلاطم شدیدی را به سامانه اعمال می‌کردند، نرخ اصابت در حافظه نهان سطح یک به شدت کاهش یافت؛ اما حافظه نهان قربانی توانست بیش از نیمی از نقصان‌های سطح یک را درون خود مهار کند. این نرخ اصابت بالا در بافر قربانی، باعث کاهش ۵۰ درصدی ترافیک ارسالی به حافظه اصلی شد و در نتیجه، زمان کل اجرای برنامه بیش از ۴۰ درصد بهبود یافت.

تحلیل و مقایسه زمان دسترسی میانگین در دو حالت تئوری و عملی نیز نشان‌دهنده بهبودهای چشمگیر

بود. اگرچه محدودیت‌های پنهان در شبکه مانند ترافیک ناشی از عملیات بازنویسی و توقف‌های ساختاری به دلیل پر شدن ثبات درخواست‌های معلق باعث می‌شوند سامانه در عمل کندتر از محاسبات ریاضی رفتار کند، اما استفاده از حافظهٔ قربانی در هر دو حالت توانست تاخیر کلی سامانه را به میزان قابل توجهی کاهش دهد و اثر مخرب تاخیر صد چرخه‌ای حافظهٔ اصلی را خنثی کند.

در پایان، این شبیه‌سازی به طور عملی نشان داد که ایجاد موازنه میان هزینهٔ سخت‌افزاری و بهبود عملکرد چگونه در طراحی معماری کامپیوتر نقش کلیدی ایفا می‌کند. با صرف یک هزینهٔ بسیار ناچیز در سطح سخت‌افزار برای ساخت یک بافر کوچک، سامانه توانست از جریمه‌های سنگین حافظه فرار کرده و پایداری عملکرد خود را در برابر بدترین و پیچیده‌ترین الگوهای دسترسی به حافظه حفظ کند.

References

- [1] Wikipedia. Memory hierarchy. https://en.wikipedia.org/wiki/Memory_hierarchy. Accessed: 2026-08-05.
- [2] GeeksforGeeks. Types of cache misses. <https://www.geeksforgeeks.org/computer-organization-architecture/types-of-cache-misses/>. Accessed: 2026-08-05.
- [3] N. P. Jouppi. Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers. In *Proceedings of the 17th Annual International Symposium on Computer Architecture*, pages 364–373, 1990.
- [4] Scalable Architecture Lab. Project akita. <https://sarchlab.org/akita>. Accessed: 2026-08-05.
- [5] L.-Y. Chang, C.-H. Suo, and Y.-Y. Liu. Counter-based victim cache hit rate optimization. In *SASIMI 2015 Proceedings*, pages 317–318, 2015.
- [6] Wikipedia. Victim cache. https://en.wikipedia.org/wiki/Victim_cache. Accessed: 2026-08-05.
- [7] Wikipedia. Average memory access time. https://en.wikipedia.org/wiki/Average_memory_access_time. Accessed: 2026-08-05.